

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Системы компьютерной обработки изображений

Лабораторная работа № 2

«Вычисление яркости в точках на поверхности»

Выполнил студент:

Стрельбицкий Илья Павлович

Группа № Р3413

Преподаватель:

Жданов Дмитрий Дмитриевич

г. Санкт-Петербург 2025

Цель

Изучить, как вычисляется яркость точек на плоскости треугольника с учетом освещения от точечных источников света и диффузно отражательных свойств поверхности.

Используемые формулы

1. «Цветная» интенсивность источника под углом к оси источника света:

$I(RGB, \vec{s}) = I_0(RGB) \cos \theta$, где $I_0(RGB)$ – «цветная» интенсивность источника света в направлении его оси $\vec{O}$, $\|\vec{O}\| = 1$, θ - угол между направлением распространения света и осью источника света, $\cos \theta$ – диаграмма излучения.

2. «Цветная» освещенность точки:

$E(RGB, \vec{P}_T) = \frac{I(RGB, \vec{s}) \cos \alpha}{R^2}$, где $I(RGB, \vec{s})$ – «цветная» интенсивность света, α - угол между направлением света и нормалью к освещаемой поверхности, R - расстояние от источника света до рассматриваемой точки.

3. Перевод локальных координат точки в плоскости в глобальные

$\vec{P}_T = \vec{P}_0 + \left(\frac{\vec{P}_1 - \vec{P}_0}{\|\vec{P}_1 - \vec{P}_0\|} \cdot x + \frac{\vec{P}_2 - \vec{P}_0}{\|\vec{P}_2 - \vec{P}_0\|} \cdot y \right)$, где x и y смещения по ребрам треугольника.

4. Вычисление вектора нормали плоскости треугольника через 3 точки:

$$\vec{N} = \frac{(\vec{P}_2 - \vec{P}_0) \times (\vec{P}_1 - \vec{P}_0)}{\|(\vec{P}_1 - \vec{P}_0) \times (\vec{P}_2 - \vec{P}_0)\|}$$

5. Вектор от точки плоскости до источника света:

$$\vec{s} = \vec{P}_T - \vec{P}_L, R^2 = \|\vec{s}\|^2, \cos \alpha = \frac{\vec{s} \cdot \vec{N}}{\|\vec{s}\|}, \cos \theta = \frac{\vec{s} \cdot \vec{O}}{\|\vec{s}\|}$$

6. Яркость точки:

$$L(RGB, \vec{P}_T, \vec{v}) = \frac{1}{\pi} \sum_{i \in \text{lights}} \left(E_i(RGB, \vec{P}_T) \cdot f(RGB, \vec{P}_T, \vec{v}, \vec{s}_i) \right), \quad \text{где}$$

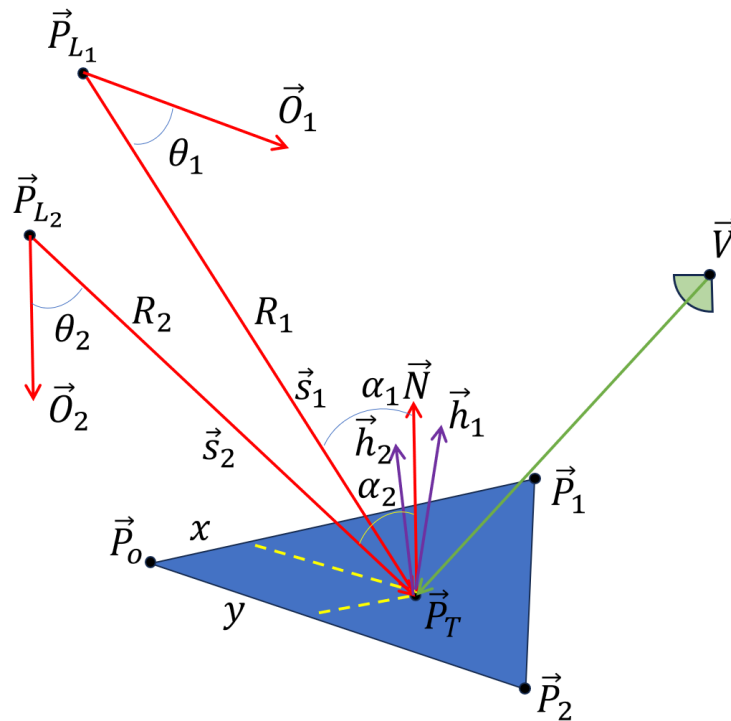
$E_i(RGB, \vec{P}_T)$ - освещённость от i -го источника света, $\vec{P}_T$ - координаты точки, $\vec{v}$ - направление на наблюдения, $\vec{s}_i$ - направление на источник света, $f(RGB, \vec{P}_T, \vec{v}, \vec{s}_i)$ - двунаправленная функция отражения (BRDF), $K(RGB)$ – «цвет» поверхности.

7. Двунаправленная функция отражения

$f(RGB, \vec{P}_T, \vec{v}, \vec{s}_i) = K(RGB) \left(k_d + k_s (\vec{h} \cdot \vec{N})^{k_e} \right)$, где k_d - коэффициент диффузного отражения поверхности, k_s - коэффициент «зеркальности» поверхности, k_e - коэффициент, определяющий ширину блика, $\vec{h}$ - средний вектор между направлениями освещения и наблюдения, $\vec{N}$ - нормаль к поверхности в точке освещения.

8. Средний вектор

$$\vec{h} = \frac{\vec{v} + \vec{s}}{\|\vec{v} + \vec{s}\|}.$$



Шаги алгоритма

1. Инициализация параметров системы

- a. Задание характеристик источников света $I_0, \vec{O}, \vec{P}_L$
- b. Определение геометрии треугольника P_0, P_1, P_2
- c. Задание параметров материала поверхности K_{RGB}, k_d, k_s, k_e
- d. Определение направления наблюдения $\vec{v}$
- e. Задание сетки точек для расчета $x_1, x_2, x_3, x_4, x_5; y_1, y_2, y_3, y_4, y_5$

2. Предварительные вычисления

- a. Ориентация нормали в сторону наблюдателя
 - Вычисляем исходную нормаль плоскости треугольника
 - Проверяем, с какой стороны находится наблюдатель относительно нормали и если наблюдатель с противоположной стороны, разворачиваем нормаль
- b. Нормализация всех входных векторов
- c. Расчет единичных векторов ребер для системы координат

3. Расчеты для каждой точки

- a. Перевод локальных координат в глобальные
- b. Расчет освещенности от каждого источника:
 - Вектор к источнику света $\vec{s}$
 - Расстояние R и его квадрат
 - $\cos \theta$ – диаграмма излучения
 - $\cos \alpha$ – угол падения
 - Проверяем то, что источник с той же стороны, что и наблюдатель, то есть если $\cos \alpha \leq 0$, то источник находится с обратной стороны или в плоскости, следовательно освещенность равна 0
 - Освещенность $E_i(RGB)$

с. Расчет яркости с учетом BRDF:

- Средний вектор $\vec{h}$
- Коэффициент бликового отражения
- BRDF функция
- Суммирование вкладов всех источников

Программа для расчета яркости

```
import numpy as np
import math

def calculate_brightness(
    lights: list[tuple[tuple[float, float, float], tuple[float, float, float], tuple[float, float, float]]],
    vect_P0: tuple[float, float, float],
    vect_P1: tuple[float, float, float],
    vect_P2: tuple[float, float, float],
    x_values: tuple[float, ...],
    y_values: tuple[float, ...],
    vect_v: tuple[float, float, float],
    K_RGB: tuple[float, float, float],
    k_d: float,
    k_s: float,
    k_e: float
) -> dict[tuple[float, float], tuple[float, float, float]]:
    """
    Расчет яркости точек на плоскости треугольника с учетом освещения и BRDF
    Параметры:
    lights - список источников света, каждый источник: (I0_RGB, vect_O, vect_P_L)
    vect_P0, vect_P1, vect_P2 - вершины треугольника
    x_values, y_values - локальные координаты точек
    vect_v - направление наблюдения (единичный вектор)
    K_RGB - цвет поверхности (R, G, B)
    k_d - коэффициент диффузного отражения
    k_s - коэффициент зеркальности
    k_e - коэффициент ширины блика
    Результат:
    results - словарь: (x, y): (L_R, L_G, L_B)
    """

    # Преобразование в numpy массивы
    P0 = np.array(vect_P0, dtype=float)
    P1 = np.array(vect_P1, dtype=float)
```

```

P2 = np.array(vect_P2, dtype=float)
v = np.array(vect_v, dtype=float)
K = np.array(K_RGB, dtype=float)
# Нормализация вектора наблюдения
v_norm = np.linalg.norm(v)
if abs(v_norm - 1.0) > 1e-10:
    v = v / v_norm
# Вычисление нормали плоскости
v1 = P1 - P0
v2 = P2 - P0
N = np.cross(v1, v2)
N_norm = np.linalg.norm(N)
if N_norm == 0:
    raise ValueError("Точки треугольника коллинеарны, нормаль не определена")
N = N / N_norm
# Если скалярное произведение вектора наблюдения и нормали отрицательное,
# значит наблюдатель находится с противоположной стороны от нормали
if np.dot(v, N) < 0:
    N = -N # Разворачиваем нормаль к наблюдателю
# Единичные векторы ребер для локальной системы координат
u1 = v1 / np.linalg.norm(v1)
u2 = v2 / np.linalg.norm(v2)
# Подготовка источников света
processed_lights = []
for I0_RGB, vect_O, vect_P_L in lights:
    I0 = np.array(I0_RGB, dtype=float)
    O = np.array(vect_O, dtype=float)
    P_L = np.array(vect_P_L, dtype=float)
    # Нормализация оси источника
    O_norm = np.linalg.norm(O)
    if abs(O_norm - 1.0) > 1e-10:
        O = O / O_norm
    processed_lights.append((I0, O, P_L))
# Генерация списка всех точек
points = [(x, y) for x in x_values for y in y_values]
# Расчет яркости для каждой точки
results = {}
for x, y in points:
    try:
        # 1. Перевод локальных координат в глобальные
        P_T = P0 + u1 * x + u2 * y
        # 2. Расчет вклада от всех источников света
        total_contribution = np.zeros(3)
        for I0, O, P_L in processed_lights:
            # Вектор от точки к источнику света
            s = P_L - P_T

```

```

R = np.linalg.norm(s)
if R < 1e-10: # Избегаем деления на ноль
    continue
# Нормализованный вектор к источнику
s_norm = s / R
# Расчет интенсивности I(РGB, vect(s))
cos_theta = np.dot(s_norm, O)
if cos_theta <= 0: # Источник не светит в эту точку
    continue
I_RGB = I0 * cos_theta
# Расчет освещенности E(РGB, vect(P_T))
cos_alpha = np.dot(s_norm, N)
if cos_alpha <= 0: # Источник с противоположной стороны или в плоскости
    continue
E_RGB = I_RGB * cos_alpha / (R * R)
# Расчет BRDF
# Средний вектор
h = v + s_norm
h_norm = np.linalg.norm(h)
if h_norm < 1e-10:
    continue
h = h / h_norm
# Коэффициент бликового отражения
specular = k_s * (abs(np.dot(h, N)) ** k_e)
# BRDF функция
f_RGB = K * (k_d + specular)
# Вклад от данного источника
total_contribution += E_RGB * f_RGB
# Итоговая яркость
L_RGB = total_contribution / math.pi
results[(x, y)] = (L_RGB[0], L_RGB[1], L_RGB[2])
except Exception as e:
    print(f"Ошибка при расчете точки ({x}, {y}): {e}")
    results[(x, y)] = (0.0, 0.0, 0.0)
return results

```

```

def calculate_illumination_single_source(
    I0_RGB: tuple[float, float, float],
    vect_O: tuple[float, float, float],
    vect_P_L: tuple[float, float, float],
    vect_P0: tuple[float, float, float],
    vect_P1: tuple[float, float, float],
    vect_P2: tuple[float, float, float],
    x_values: tuple[float, ...],
    y_values: tuple[float, ...]

```

```
) -> dict[tuple[float, float], tuple[float, float, float]]:
```

```
"""
```

```
Расчет освещенности от одного источника (из предыдущей лабораторной)
```

```
"""
```

```
# Преобразование в numpy массивы
```

```
I0 = np.array(I0_RGB, dtype=float)
```

```
O = np.array(vect_O, dtype=float)
```

```
P_L = np.array(vect_P_L, dtype=float)
```

```
P0 = np.array(vect_P0, dtype=float)
```

```
P1 = np.array(vect_P1, dtype=float)
```

```
P2 = np.array(vect_P2, dtype=float)
```

```
# Нормализация оси источника
```

```
O_norm = np.linalg.norm(O)
```

```
if abs(O_norm - 1.0) > 1e-10:
```

```
    O = O / O_norm
```

```
# Вычисление нормали плоскости
```

```
v1 = P1 - P0
```

```
v2 = P2 - P0
```

```
N = np.cross(v1, v2)
```

```
N_norm = np.linalg.norm(N)
```

```
if N_norm == 0:
```

```
    raise ValueError("Точки треугольника коллинеарны, нормаль не определена")
```

```
N = N / N_norm
```

```
# Единичные векторы ребер
```

```
u1 = v1 / np.linalg.norm(v1)
```

```
u2 = v2 / np.linalg.norm(v2)
```

```
points = [(x, y) for x in x_values for y in y_values]
```

```
results = {}
```

```
for x, y in points:
```

```
    try:
```

```
        # Перевод локальных координат в глобальные
```

```
        P_T = P0 + u1 * x + u2 * y
```

```
        # Вектор к источнику света
```

```
        s = P_L - P_T
```

```
        R = np.linalg.norm(s)
```

```
        if R < 1e-10:
```

```
            results[(x, y)] = (0.0, 0.0, 0.0)
```

```
            continue
```

```
        # Расчет интенсивности
```

```
        cos_theta = np.dot(s, O) / R
```

```
        I_RGB = I0 * abs(cos_theta)
```

```
        # Расчет освещенности
```

```
        cos_alpha = np.dot(s, N) / R
```

```
        E_RGB = I_RGB * abs(cos_alpha) / (R * R)
```

```
        results[(x, y)] = (E_RGB[0], E_RGB[1], E_RGB[2])
```

```
    except Exception as e:
```



```

        print(f"Ошибка при расчете точки ({x}, {y}): {e}")
        results[(x, y)] = (0.0, 0.0, 0.0)
    return results

def print_results_table(
    results: dict[tuple[float, float], tuple[float, float, float]],
    x_values: tuple[float, ...],
    y_values: tuple[float, ...],
    title: str = "Результаты"
) -> None:
    """Красивое отображение результатов в виде таблицы"""
    print(f"{title}:")
    num_cols = len(x_values)
    col_width = 19
    # Верхняя граница таблицы
    top_border = "+" + "-" * 9 + "+" + ("-" * (col_width + 2) + "+") * num_cols
    print(top_border)
    # Заголовок с x координатами
    header = " | y\\x |"
    for x in x_values:
        header += f" {x:^{col_width}.2f} |"
    print(header)
    print(top_border)
    # Отображение данных
    for y in y_values:
        row = f" | {y:>6.2f} |"
        for x in x_values:
            values = results.get((x, y), (0, 0, 0))
            # Форматируем RGB значения
            rgb_str = f"{values[0]:.3f}, {values[1]:.3f}, {values[2]:.3f}"
            row += f" {rgb_str:^{col_width}} |"
        print(row)
    print(top_border)

def main() -> None:
    # Входные данные для двух источников света
    # Источник 1
    I0_1_RGB = (80.0, 60.0, 40.0) # Более теплый свет
    vect_O_1 = (0.1, -0.2, 1.0) # Ось первого источника
    vect_P_L_1 = (1.0, 1.0, 4.0) # Положение первого источника
    # Источник 2
    I0_2_RGB = (40.0, 60.0, 80.0) # Более холодный свет
    vect_O_2 = (-0.1, -0.3, 1.0) # Ось второго источника
    vect_P_L_2 = (-1.0, 2.0, 3.0) # Положение второго источника
    # Геометрия треугольника

```

```

vect_P0 = (0.0, 0.0, 0.0)
vect_P1 = (2.0, 0.0, 0.0)
vect_P2 = (0.0, 2.0, 0.0)
# Сетка точек
x_1, x_2, x_3, x_4, x_5 = 0.0, 0.5, 1.0, 1.5, 2.0
y_1, y_2, y_3, y_4, y_5 = 0.0, 0.4, 0.8, 1.2, 1.6
x_values = (x_1, x_2, x_3, x_4, x_5)
y_values = (y_1, y_2, y_3, y_4, y_5)
# Параметры наблюдения и материала
vect_v = (0.0, 0.0, 1.0) # Направление наблюдения (сверху)
K_RGB = (0.8, 0.6, 0.4) # Цвет поверхности (коричневатый)
k_d = 0.7 # Коэффициент диффузного отражения
k_s = 0.3 # Коэффициент зеркальности
k_e = 8.0 # Коэффициент ширины блика
# Вывод входных параметров
print("Входные параметры:")
print(f"1. I_O_1(RGB) = {I_O_1_RGB}; I_O_2(RGB) = {I_O_2_RGB}")
print(f"2. vect(O1) = {vect_O_1}; vect(O2) = {vect_O_2}")
print(f"3. vect(P_L_1) = {vect_P_L_1}; vect(P_L_2) = {vect_P_L_2}")
print(f"4. vect(P0) = {vect_P0}")
print(f"5. vect(P1) = {vect_P1}")
print(f"6. vect(P2) = {vect_P2}")
print(f"7. x1={{x_1}}, y1={{y_1}}; x2={{x_2}}, y2={{y_2}}; x3={{x_3}}, y3={{y_3}}; x4={{x_4}}, y4={{y_4}}; x5={{x_5}}, y5={{y_5}}")
print(f"8. vect(V) = {vect_v}")
print(f"9. K(RGB) = {K_RGB}")
print(f"10. k_d = {k_d}")
print(f"11. k_s = {k_s}")
print(f"12. k_e = {k_e}")
# Расчет освещенности от первого источника
print()
print("Расчет освещенности от первого источника:")
results_E1 = calculate_illumination_single_source(
    I_O_1_RGB, vect_O_1, vect_P_L_1, vect_P0, vect_P1, vect_P2, x_values, y_values
)
print_results_table(
    results_E1, x_values, y_values, "Освещенность E1(RGB,vect(P)) от первого источника"
)
# Расчет освещенности от второго источника
print()
print("Расчет освещенности от второго источника:")
results_E2 = calculate_illumination_single_source(
    I_O_2_RGB, vect_O_2, vect_P_L_2, vect_P0, vect_P1, vect_P2, x_values, y_values
)
print_results_table(
    results_E2, x_values, y_values, "Освещенность E2(RGB,vect(P)) от второго источника"
)

```

```

)
# Расчет яркости с учетом обоих источников и BRDF
print()
print("Расчет яркости с учетом BRDF и всех источников:")
# Список всех источников света
lights = [
    (I0_1_RGB, vect_O_1, vect_P_L_1),
    (I0_2_RGB, vect_O_2, vect_P_L_2)
]
results_L = calculate_brightness(
    lights, vect_P0, vect_P1, vect_P2, x_values, y_values,
    vect_v, K_RGB, k_d, k_s, k_e
)
print_results_table(
    results_L, x_values, y_values, "Яркость L(RGB,vect(P_T),vect(v))"
)
# Дополнительная информация
print()
print("Дополнительная информация для проверки:")
# Нормаль плоскости
v1 = np.array(vect_P1) - np.array(vect_P0)
v2 = np.array(vect_P2) - np.array(vect_P0)
N = np.cross(v1, v2)
N_norm = N / np.linalg.norm(N)
print(f"Нормаль плоскости: ({N_norm[0]:.3f}, {N_norm[1]:.3f}, {N_norm[2]:.3f})")
# Проверка входных векторов
O1_norm = np.linalg.norm(vect_O_1)
O2_norm = np.linalg.norm(vect_O_2)
v_norm = np.linalg.norm(vect_v)
print(f"Длина vect(O1): {O1_norm:.3f}, vect(O2): {O2_norm:.3f}, vect(v): {v_norm:.3f}")

if __name__ == "__main__":
    main()

```

Вывод программы в консоль:

```
second master Current File Run main.py
"D:\university\methods obr image\second\.venv\Scripts\python.exe" "D:\university\methods obr image\second\main.py"
Входные параметры:
1. I_0_1(RGB) = (80.0, 60.0, 40.0); I_0_2(RGB) = (40.0, 60.0, 80.0)
2. vect(O_1) = (0.1, -0.2, 1.0); vect(O_2) = (-0.1, -0.3, 1.0)
3. vect(P_L_1) = (1.0, 1.0, 4.0); vect(P_L_2) = (-1.0, 2.0, 3.0)
4. vect(P_0) = (0.0, 0.0, 0.0)
5. vect(P_1) = (2.0, 0.0, 0.0)
6. vect(P_2) = (0.0, 2.0, 0.0)
7. x_1=0.0, y_1=0.0; x_2=0.5, y_2=0.4; x_3=1.0, y_3=0.8; x_4=1.5, y_4=1.2; x_5=2.0, y_5=1.6
8. vect(V) = (0.0, 0.0, 1.0)
9. K(RGB) = (0.8, 0.6, 0.4)
10. k_d = 0.7
11. k_s = 0.3
12. k_e = 8.0

Расчет освещенности от первого источника:
Освещенность E_1(RGB,vect(P)) от первого источника:
+-----+-----+-----+-----+-----+
| y\x | 0.00 | 0.50 | 1.00 | 1.50 | 2.00 |
+-----+-----+-----+-----+-----+
| 0.00 | 3.759, 2.819, 1.880 | 4.041, 3.030, 2.020 | 4.106, 3.080, 2.053 | 3.936, 2.952, 1.968 | 3.566, 2.675, 1.783 |
+-----+-----+-----+-----+-----+
| 0.40 | 4.124, 3.093, 2.062 | 4.448, 3.336, 2.224 | 4.527, 3.395, 2.264 | 4.335, 3.251, 2.168 | 3.917, 2.938, 1.958 |
+-----+-----+-----+-----+-----+

second main.py 260:54 CRLF UTF-8 4 spaces Python 3.13 (second)
```

```
second master Current File Run main.py
Расчет освещенности от первого источника:
Освещенность E_1(RGB,vect(P)) от первого источника:
+-----+-----+-----+-----+-----+
| y\x | 0.00 | 0.50 | 1.00 | 1.50 | 2.00 |
+-----+-----+-----+-----+-----+
| 0.00 | 3.759, 2.819, 1.880 | 4.041, 3.030, 2.020 | 4.106, 3.080, 2.053 | 3.936, 2.952, 1.968 | 3.566, 2.675, 1.783 |
+-----+-----+-----+-----+-----+
| 0.40 | 4.124, 3.093, 2.062 | 4.448, 3.336, 2.224 | 4.527, 3.395, 2.264 | 4.335, 3.251, 2.168 | 3.917, 2.938, 1.958 |
+-----+-----+-----+-----+-----+
| 0.80 | 4.367, 3.275, 2.183 | 4.719, 3.539, 2.360 | 4.807, 3.605, 2.403 | 4.601, 3.451, 2.301 | 4.151, 3.114, 2.076 |
+-----+-----+-----+-----+-----+
| 1.20 | 4.453, 3.339, 2.226 | 4.813, 3.610, 2.407 | 4.904, 3.678, 2.452 | 4.696, 3.522, 2.348 | 4.238, 3.178, 2.119 |
+-----+-----+-----+-----+-----+
| 1.60 | 4.373, 3.280, 2.186 | 4.720, 3.540, 2.360 | 4.807, 3.605, 2.404 | 4.607, 3.455, 2.303 | 4.166, 3.124, 2.083 |
+-----+-----+-----+-----+-----+

Расчет освещенности от второго источника:
Освещенность E_2(RGB,vect(P)) от второго источника:
+-----+-----+-----+-----+-----+
| y\x | 0.00 | 0.50 | 1.00 | 1.50 | 2.00 |
+-----+-----+-----+-----+-----+
| 0.00 | 1.459, 2.189, 2.919 | 1.255, 1.882, 2.509 | 1.029, 1.544, 2.059 | 0.818, 1.227, 1.636 | 0.638, 0.957, 1.277 |
+-----+-----+-----+-----+-----+

second main.py 260:54 CRLF UTF-8 4 spaces Python 3.13 (second)
```

```

Project Alt+1
main.py x
Run main x
:
Расчет освещенности от второго источника:
Освещенность E2(RGB, vect(P)) от второго источника:
+-----+
| y\x | 0.00 | 0.50 | 1.00 | 1.50 | 2.00 |
+-----+
| 0.00 | 1.459, 2.189, 2.919 | 1.255, 1.882, 2.509 | 1.029, 1.544, 2.059 | 0.818, 1.227, 1.636 | 0.638, 0.957, 1.277 |
+-----+
| 0.40 | 1.908, 2.850, 3.800 | 1.602, 2.403, 3.204 | 1.285, 1.928, 2.571 | 0.999, 1.499, 1.998 | 0.763, 1.145, 1.527 |
+-----+
| 0.80 | 2.395, 3.593, 4.791 | 1.982, 2.973, 3.965 | 1.558, 2.338, 3.117 | 1.187, 1.781, 2.374 | 0.890, 1.335, 1.780 |
+-----+
| 1.20 | 2.890, 4.336, 5.781 | 2.355, 3.533, 4.710 | 1.820, 2.730, 3.641 | 1.364, 2.046, 2.728 | 1.008, 1.511, 2.015 |
+-----+
| 1.60 | 3.303, 4.955, 6.606 | 2.663, 3.994, 5.326 | 2.035, 3.052, 4.070 | 1.508, 2.262, 3.016 | 1.103, 1.655, 2.207 |
+-----+

Расчет яркости с учетом BRDF и всех источников:
Яркость L(RGB, vect(P_T), vect(v)):
+-----+
| y\x | 0.00 | 0.50 | 1.00 | 1.50 | 2.00 |
+-----+
| 0.00 | 1.259, 0.896, 0.565 | 1.289, 0.884, 0.534 | 1.255, 0.834, 0.484 | 1.158, 0.751, 0.422 | 1.014, 0.647, 0.355 |
+-----+

```

```

Project
main.py x
Run main x
:
Расчет яркости с учетом BRDF и всех источников:
Яркость L(RGB, vect(P_T), vect(v)):
+-----+
| y\x | 0.00 | 0.50 | 1.00 | 1.50 | 2.00 |
+-----+
| 0.00 | 1.259, 0.896, 0.565 | 1.289, 0.884, 0.534 | 1.255, 0.834, 0.484 | 1.158, 0.751, 0.422 | 1.014, 0.647, 0.355 |
+-----+
| 0.40 | 1.470, 1.077, 0.701 | 1.487, 1.043, 0.647 | 1.433, 0.968, 0.574 | 1.311, 0.861, 0.492 | 1.138, 0.732, 0.407 |
+-----+
| 0.80 | 1.664, 1.258, 0.845 | 1.658, 1.192, 0.761 | 1.577, 1.086, 0.660 | 1.428, 0.951, 0.554 | 1.231, 0.800, 0.452 |
+-----+
| 1.20 | 1.816, 1.417, 0.981 | 1.776, 1.313, 0.862 | 1.666, 1.172, 0.730 | 1.494, 1.012, 0.602 | 1.279, 0.843, 0.484 |
+-----+
| 1.60 | 1.900, 1.526, 1.085 | 1.825, 1.384, 0.933 | 1.687, 1.214, 0.775 | 1.499, 1.034, 0.629 | 1.277, 0.854, 0.500 |
+-----+

Дополнительная информация для проверки:
Нормаль плоскости: (0.000, 0.000, 1.000)
Длина vect(0_1): 1.025, vect(0_2): 1.049, vect(v): 1.000

Process finished with exit code 0

```

Пример входных данных и результаты

- $I_{0_1}(RGB) = (80.0, 60.0, 40.0)$
 $I_{0_2}(RGB) = (40.0, 60.0, 80.0)$
- $\vec{0}_1 = (0.1, -0.2, 1.0)$
 $\vec{0}_2 = (-0.1, -0.3, 1.0)$

3. $\vec{P}_{L_1} = (1.0, 1.0, 4.0)$
 $\vec{P}_{L_2} = (-1.0, 2.0, 3.0)$
4. $\vec{P}_0 = (0.0, 0.0, 0.0)$
5. $\vec{P}_1 = (2.0, 0.0, 0.0)$
6. $\vec{P}_2 = (0.0, 2.0, 0.0)$
7. $x_1 = 0.0, y_1 = 0.0$
 $x_2 = 0.5, y_2 = 0.4$
 $x_3 = 1.0, y_3 = 0.8$
 $x_4 = 1.5, y_4 = 1.2$
 $x_5 = 2.0, y_5 = 1.6$
8. $\vec{V} = (0.0, 0.0, 1.0)$
9. $K(RGB) = (0.8, 0.6, 0.4)$
10. $k_d = 0.7$
11. $k_s = 0.3$
12. $k_e = 8.0$

Вычисленные значения освещенности $E_1(RGB, \vec{P}_T)$ для точек, заданных локальными координатами:

$x \backslash y$	0.00	0.50	1.00	1.50	2.00
0.00	3.759, 2.819, 1.880	4.041, 3.030, 2.020	4.106, 3.080, 2.053	3.936, 2.952, 1.968	3.566, 2.675, 1.783
0.40	4.124, 3.093, 2.062	4.448, 3.336, 2.224	4.527, 3.395, 2.264	4.335, 3.251, 2.168	3.917, 2.938, 1.958
0.80	4.367, 3.275, 2.183	4.719, 3.539, 2.360	4.807, 3.605, 2.403	4.601, 3.451, 2.301	4.151, 3.114, 2.076
1.20	4.453, 3.339, 2.226	4.813, 3.610, 2.407	4.904, 3.678, 2.452	4.696, 3.522, 2.348	4.238, 3.178, 2.119
1.60	4.373, 3.280, 2.186	4.720, 3.540, 2.360	4.807, 3.605, 2.404	4.607, 3.455, 2.303	4.166, 3.124, 2.083

Вычисленные значения освещенности $E_2 \left((RGB, \vec{P}_T) \right)$ для тех же точек, заданных локальными координатами:

$x \backslash y$	0.00	0.50	1.00	1.50	2.00
0.00	1.459, 2.189, 2.919	1.255, 1.882, 2.509	1.029, 1.544, 2.059	0.818, 1.227, 1.636	0.638, 0.957, 1.277
0.40	1.900, 2.850, 3.800	1.602, 2.403, 3.204	1.285, 1.928, 2.571	0.999, 1.499, 1.998	0.763, 1.145, 1.527
0.80	2.395, 3.593, 4.791	1.982, 2.973, 3.965	1.558, 2.338, 3.117	1.187, 1.781, 2.374	0.890, 1.335, 1.780
1.20	2.890, 4.336, 5.781	2.355, 3.533, 4.71	1.820, 2.730, 3.641	1.364, 2.046, 2.728	1.008, 1.511, 2.015
1.60	3.303, 4.955, 6.606	2.663, 3.994, 5.326	2.035, 3.052, 4.070	1.508, 2.262, 3.016	1.103, 1.655, 2.207

Вычисленные значения яркостей $L(RGB, \vec{P}_T, \vec{v})$ для тех же точек на плоскости с заданными условиями наблюдения

$x \backslash y$	0.00	0.50	1.00	1.50	2.00
0.00	1.259, 0.896, 0.565	1.289, 0.884, 0.534	1.255, 0.834, 0.484	1.158, 0.751, 0.422	1.014, 0.647, 0.355
0.40	1.470, 1.077, 0.701	1.487, 1.043, 0.647	1.433, 0.968, 0.574	1.311, 0.861, 0.492	1.138, 0.732, 0.407
0.80	1.664, 1.258, 0.845	1.658, 1.192, 0.761	1.577, 1.086, 0.660	1.428, 0.951, 0.554	1.231, 0.800, 0.452
1.20	1.816, 1.417, 0.981	1.776, 1.313, 0.862	1.666, 1.172, 0.730	1.494, 1.012, 0.602	1.279, 0.843, 0.484
1.60	1.900, 1.526, 1.085	1.825, 1.384, 0.933	1.687, 1.214, 0.775	1.499, 1.034, 0.629	1.277, 0.854, 0.500

Вывод

В данной работе был изучен и реализован алгоритм вычисления яркости точки на плоскости с учетом освещенности от нескольких источников света. Используя заданные входные данные, программа корректно вычисляет значения освещенности и яркости для различных точек плоскости, учитывая направленность света и свойства поверхности.